



Documentación Técnica

CoopDoc

Editor de Texto Colaborativo de Escritorio (Bloc de Notas Colaborativo)

De la Garza Basurto Fernando
Delgadillo Escartín Eduardo Abraham
Huerta Cruz Jenifer Naomi
Rodríguez Flores María José
Serrano Cid Sebastián

Sistemas Operativos
Ríos Olivares César Antonio

08 / 06 / 26

1. Información General del Proyecto

1.1. Nombre del proyecto

CoopDoc — Editor de Texto Colaborativo de Escritorio (Bloc de Notas Colaborativo).

1.2. Objetivo general y objetivos específicos

Objetivo general: Desarrollar una aplicación de escritorio para Windows que permita explorar conceptos de gestión de recursos y manejo de múltiples usuarios trabajando de forma simultánea, materializados en un editor de texto colaborativo en tiempo real construido en Python sobre una arquitectura cliente-servidor.

Objetivos específicos:

- Implementar la autenticación de usuarios (registro e inicio de sesión) con validación estricta de datos, aplicada tanto en el cliente como en el servidor.
- Construir una arquitectura cliente-servidor que conecte un cliente de escritorio (tkinter) con un servicio backend (FastAPI) y una base de datos (SQLite), empleando HTTP/REST para las operaciones estándar y WebSockets para la sincronización en tiempo real.
- Desarrollar un lienzo de edición que soporte las operaciones básicas de texto: escribir, borrar, aplicar negritas e itálicas e insertar saltos de línea.
- Permitir la importación y exportación de documentos en formato de texto plano (.txt).
- Gestionar la colaboración multiusuario mediante la invitación y revocación de accesos, así como la sincronización concurrente del contenido.
- Persistir de forma confiable los datos de usuarios, documentos y permisos en una base de datos SQLite.

1.3. Descripción breve del producto

CoopDoc es una aplicación de escritorio multiusuario, conceptualmente similar al Bloc de notas de Windows, que permite a varias personas crear una cuenta, iniciar sesión y editar documentos de texto de manera simultánea. El sistema ofrece un flujo de autenticación seguro, un lienzo de edición con formato básico, un esquema de permisos basado en roles (propietario y editor) y la capacidad de exportar el trabajo a archivos .txt. Su propósito didáctico es evidenciar la gestión concurrente de recursos y el acceso multiusuario propios de la materia de Sistemas Operativos.

1.4. Justificación y alcance

Justificación: El proyecto funciona como evidencia integradora de las competencias de Sistemas Operativos, demostrando la capacidad de coordinar múltiples clientes concurrentes, administrar el acceso compartido a un recurso (el documento) y mantener la integridad de los datos persistidos, todo ello dentro de una aplicación funcional de extremo a extremo.

Alcance (dentro del proyecto):

- Registro e inicio de sesión de usuarios con validación de nombre, correo y contraseña.
- Lienzo de edición con escritura, borrado, negritas, itálicas y saltos de línea.
- Importación y exportación de documentos en formato .txt.
- Colaboración: invitación y revocación de colaboradores y sincronización del texto en tiempo real entre clientes activos.

Fuera del alcance (posibles implementaciones a futuro):

- Elementos de edición adicionales: listas ordenadas y no ordenadas, texto resaltado y color de texto.
- Soporte para el formato Markdown (.md), permitiendo crear, editar, importar y exportar dicho formato.
- Un sistema de organización de documentos más completo.

1.5. Equipo de desarrollo

El proyecto se organizó simulando un marco de trabajo Scrum simplificado, con la siguiente distribución de roles:

- **Product Owner:** Ríos Olivares César Antonio.
- **Scrum Master:** Delgadillo Escartín Eduardo Abraham.
- **Development Team:** De la Garza Basurto Fernando, Huerta Cruz Jenifer Naomi, Rodríguez Flores María José y Serrano Cid Sebastián.

1.6. Cronograma general (Roadmap Ágil)

El desarrollo se ejecutó siguiendo una metodología ágil incremental, organizada en sprints semanales. A continuación se detallan las actividades planificadas en cada iteración.

Sprint 1 — Planeación y diseño (8 – 15 de mayo)

- Planeación: definir y documentar el cronograma detallado del proyecto.
- Planeación: diseñar el modelado de la aplicación mediante diagramas UML.
- Configuración: crear y configurar el repositorio en Git y GitHub.
- Diseño UX/UI: crear los wireframes y el diseño en Figma de las pantallas de inicio de sesión y registro.
- Diseño UX/UI: crear los wireframes y el diseño en Figma del lienzo del editor.

Sprint 2 — Backend y autenticación (15 – 22 de mayo)

- Backend: crear la base de datos y la arquitectura del backend para usuarios.
- Autenticación: desarrollar la funcionalidad de creación de cuenta.
- Autenticación: desarrollar la funcionalidad de inicio de sesión y su validación.

Sprint 3 — Editor base (22 – 29 de mayo)

- Core Editor: implementar el lienzo principal para escribir y borrar texto.
- Core Editor: añadir las funciones de formato (negritas e itálicas).

Sprint 4 — Archivos y colaboración (29 de mayo – 5 de junio)

- Archivos: desarrollar la función para abrir y guardar archivos en formato .txt.
- Colaboración: implementar la lógica de base de datos para invitar y eliminar colaboradores.
- Colaboración: construir la infraestructura en tiempo real para la edición simultánea.

Sprint 6 — Cierre y documentación (5 – 12 de junio)

- Cierre: redactar el reporte de pruebas, errores y correcciones.
- Documentación: elaborar el manual del usuario.
- Documentación: redactar la documentación técnica (aplicación y código).
- Cierre: compilar y dar formato al reporte académico final.

2. Documentación Técnica

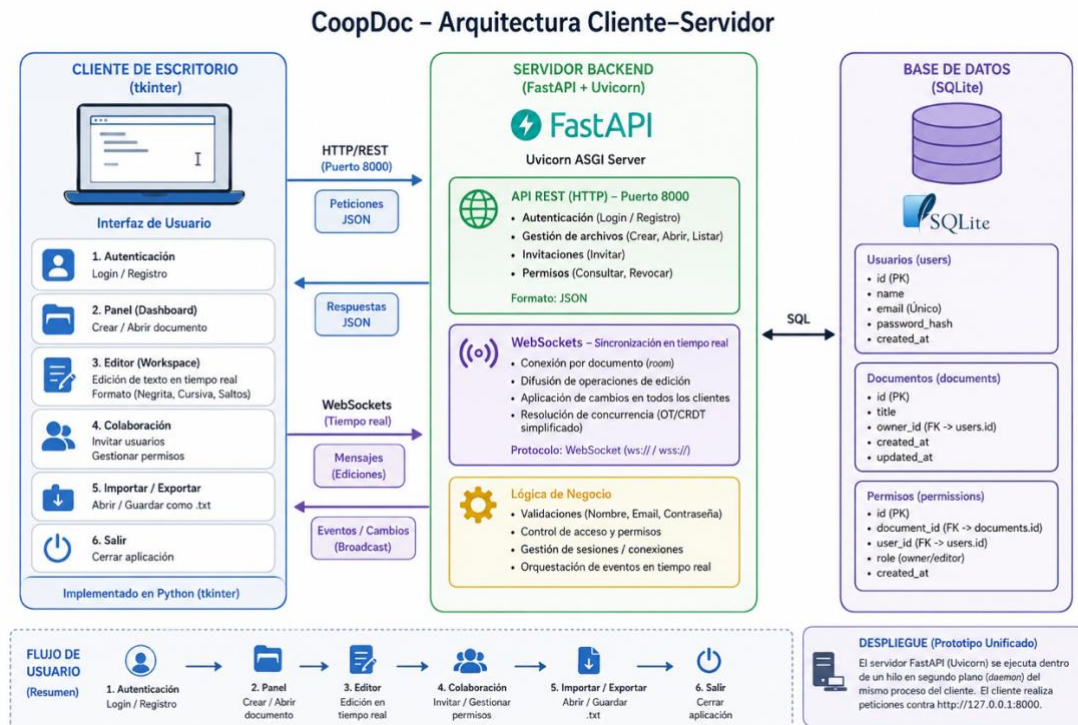
2.1. Diagrama de arquitectura general

CoopDoc implementa un modelo cliente-servidor. El cliente de escritorio (tkinter) presenta la interfaz y se comunica con un servicio backend (FastAPI servido por Uvicorn) a través de dos protocolos complementarios:

- **HTTP/REST (puerto 8000):** utilizado para las operaciones estándar como autenticación, gestión de archivos e invitaciones, mediante cargas útiles en formato JSON.
- **WebSockets:** utilizado para la sincronización del texto en tiempo real entre todos los clientes que editan el mismo documento.

La persistencia se delega a una base de datos SQLite, que almacena los registros de usuarios, los documentos y los permisos de acceso. En el prototipo unificado, el servidor FastAPI se ejecuta dentro de un hilo en segundo plano (daemon) del mismo proceso, y el cliente realiza sus peticiones contra la dirección local mediante la librería requests; de este modo, toda la aplicación se entrega como un único archivo ejecutable.

El flujo de usuario estándar es el siguiente: el usuario abre la aplicación y aterriza en la interfaz de autenticación; inicia sesión o crea una cuenta; accede a un panel con opciones para crear o abrir un documento; entra al lienzo de edición; colabora en tiempo real e invita o gestiona colaboradores; y, finalmente, exporta o importa documentos .txt antes de cerrar la aplicación.



2.2. Tecnologías y herramientas

Componente	Tecnología / Herramienta
Lenguaje	Python 3
Interfaz gráfica (cliente)	tkinter (con tkinter.font y messagebox)
Servidor backend	FastAPI sobre el servidor ASGI Uvicorn
Validación de datos	Pydantic (field_validator, model_validator)
Base de datos	SQLite (módulo sqlite3)
Cliente HTTP	requests
Seguridad / utilidades	hashlib (SHA-256), re (expresiones regulares)
Control de versiones	Git y GitHub
Tipografía (guía de diseño)	Rubik para la interfaz; Open Sans para el lienzo de texto

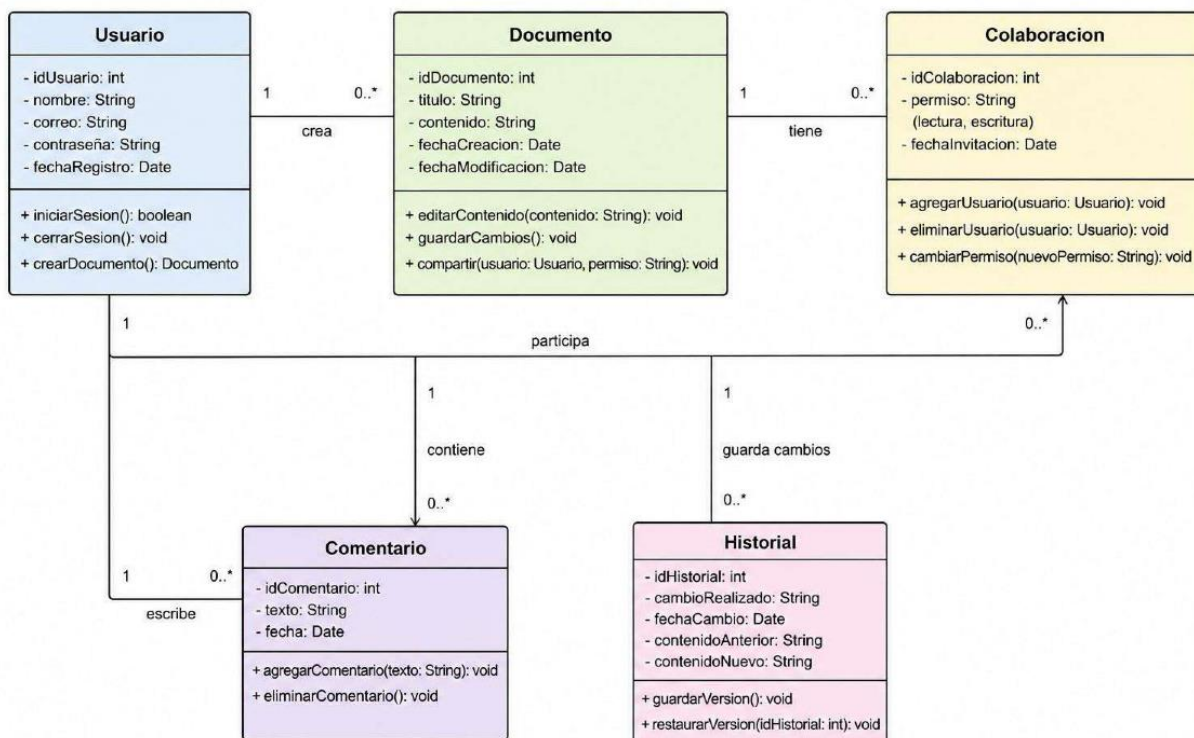
Repositorio del proyecto: <https://github.com/laloescar/collaborative-notepad>

2.3. Justificación técnica

- **tkinter:** forma parte de la librería estándar de Python, no requiere dependencias externas y se adapta al enfoque “Windows-first” del proyecto.
- **FastAPI:** ofrece validación declarativa con Pydantic, manejo asíncrono de peticiones, soporte nativo de WebSockets y una definición de endpoints REST sencilla.
- **SQLite:** base de datos embebida y sin servidor, ideal para almacenar usuarios, documentos y permisos con una huella mínima.
- **WebSockets:** se eligieron sobre SSE por permitir comunicación bidireccional, necesaria para reflejar la edición concurrente en todos los clientes.
- **check_same_thread=False:** se habilita porque FastAPI atiende peticiones concurrentes en múltiples hilos, mientras que SQLite restringe por defecto la conexión al hilo que la creó (concepto de concurrencia de Sistemas Operativos).
- **Servidor embebido en un hilo daemon:** permite distribuir cliente y servidor en un solo proceso y un único archivo, simplificando la ejecución del prototipo.
- **SHA-256 (hashlib):** proporciona un hash de contraseñas sin dependencias externas. Nota técnica: en un entorno de producción se recomendaría bcrypt o argon2 con sal por usuario.

2.4. Modelo conceptual de clases (UML)

El diseño orientado a objetos del sistema se plasmó en un diagrama UML conceptual. Este modelo describe las entidades principales y sus relaciones, y sirvió como guía para la implementación.



- **Usuario:** atributos idUsuario, nombre, correo, contraseña y fechaRegistro; operaciones iniciarSesion(), cerrarSesion() y crearDocumento().
- **Documento:** atributos idDocumento, titulo, contenido, fechaCreacion y fechaModificacion; operaciones editarContenido(), guardarCambios() y compartir().
- **Colaboracion:** atributos idColaboracion, permiso (lectura/escritura) y fechaInvitacion; operaciones agregarUsuario(), eliminarUsuario() y cambiarPermiso().
- **Comentario:** atributos idComentario, texto y fecha; operaciones agregarComentario() y eliminarComentario().
- **Historial:** atributos idHistorial, cambioRealizado, fechaCambio, contenidoAnterior y contenidoNuevo; operaciones guardarVersion() y restaurarVersion().

Relaciones principales: un Usuario crea cero o más Documentos; un Documento tiene cero o más Colaboraciones; un Usuario escribe cero o más Comentarios; y un Documento guarda cambios en cero o más registros de Historial.

Nota de implementación: El alcance entregado materializa las entidades Usuario, Documento y Colaboración (esta última a través de la tabla de permisos). Las entidades Comentario e Historial forman parte del modelo conceptual de diseño y se contemplan como base para iteraciones futuras.

2.5. Modelo de datos implementado (SQLite)

La base de datos editor_backend.db se inicializa automáticamente al arrancar el servidor y consta de tres tablas.

Tabla users

Campo	Tipo	Restricciones
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
name	TEXT	NOT NULL
date_of_birth	TEXT	Formato dd/mm/aaaa
email	TEXT	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL (SHA-256)

Tabla documents

Campo	Tipo	Restricciones
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
title	TEXT	NOT NULL
content	TEXT	DEFAULT "
owner_id	INTEGER	FOREIGN KEY → users(id)

Tabla permissions

Campo	Tipo	Restricciones
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
document_id	INTEGER	FOREIGN KEY → documents(id)
user_id	INTEGER	FOREIGN KEY → users(id)
role	TEXT	NOT NULL ('owner' 'editor')
(restricción)	—	UNIQUE(document_id, user_id)

2.6. Endpoints del servicio (API)

Método y ruta	Descripción	Validación / Permiso
GET /	Sonda de salud (health check); el cliente la usa para esperar a que el servidor esté listo.	—
POST /register	Registra un nuevo usuario.	Validación Pydantic; 400 si el correo ya existe.
POST /login	Autentica por correo y contraseña.	401 si las credenciales son inválidas.
POST /documents	Crea un documento y asigna al creador como 'owner'.	Requiere requester_id.
GET /documents/{id}	Obtiene el título y el contenido de un documento.	403 si no se tiene acceso.

Método y ruta	Descripción	Validación / Permiso
POST /invite	Invita a un usuario (por correo) como 'editor'.	Requiere acceso previo al documento (403).
DELETE /revoke	Revoca el acceso de un usuario.	Solo el 'owner' (403); no puede auto-revocarse (400).
WS /ws/{doc_id}/{user_id}	Canal WebSocket para la edición en tiempo real.	Verifica permiso antes de aceptar la conexión.

2.7. Estándares de codificación

- **PEP-8:** uso de snake_case para variables y funciones, y CamelCase para clases.
- **Docstrings:** documentación en línea para clases y métodos relevantes.
- **Tokens de diseño:** todos los colores se definen con códigos hexadecimales (APP_BACKGROUND, CANVAS_BACKGROUND, PRIMARY_ACCENT, HOVER_ACCENT, TEXT_PRIMARY); se evitan los nombres de color en cadena.
- **Tipografía:** Rubik para la interfaz y Open Sans (12 pt) de forma exclusiva para el lienzo de edición.
- **Validación dual:** las reglas de integridad de datos se aplican con expresiones regulares tanto en el cliente como en el servidor.

2.8. Estructura del proyecto

```

/collaborative-notepad
|
|—— archivos de aplicación
|
|   |—— coopdoc_app.py
|   |—— database.db
|
|—— referencias
|
|   |—— archivos de referencia del programa

```

Inicialmente se tenían tres archivos .py separados (backend, inicio de sesión y registro) y se consolidaron posteriormente en el archivo coopdoc_app.py mediante un commit de integración, conformando el primer prototipo funcional de extremo a extremo.

2.9. Instalación

1. Instalar Python 3 en el sistema.
2. Instalar las dependencias del proyecto:

```
pip install fastapi uvicorn requests pydantic
```

3. Ejecutar la aplicación:

```
python3 coopdoc_app.py
```

Al iniciar, el servidor FastAPI se levanta en un hilo en segundo plano; la aplicación espera a que responda la sonda de salud y, una vez lista, muestra la interfaz gráfica del cliente.

3. Pruebas y Control de Calidad

3.1. Plan de pruebas

Se definió una estrategia de pruebas centrada en validar la lógica de negocio y la integración cliente-servidor sin depender de la interacción manual. La estrategia contempla cuatro frentes:

- **Pruebas de validación:** verifican las reglas estrictas de nombre, correo y contraseña en el registro.
- **Pruebas de integración:** comprueban que el registro y el inicio de sesión escriben y recuperan correctamente la información en SQLite a través de la API.
- **Pruebas de interfaz:** validan manualmente el comportamiento de los campos, los marcadores de posición y el alternado de visibilidad de contraseña.
- **Pruebas de concurrencia:** verifican la sincronización del texto en tiempo real entre dos o más clientes conectados al mismo documento.

Como herramienta recomendada se contempla pytest junto con el TestClient de FastAPI para automatizar las pruebas de los endpoints.

3.2. Casos de prueba documentados

ID	Caso	Pasos	Resultado esperado	Estado
CP-01	Registro válido	Completar el formulario con datos válidos y enviar.	HTTP 201; el usuario queda almacenado en la base de datos.	Aprobado
CP-02	Nombre inválido	Ingresar un nombre con números o símbolos.	Se muestra un error de validación y no se envía.	Aprobado
CP-03	Correo inválido	Ingresar un correo sin @ o con puntos consecutivos.	Se muestra un error de validación de correo.	Aprobado
CP-04	Contraseña débil	Ingresar una contraseña sin mayúscula, número o símbolo.	Se muestra el requisito incumplido.	Aprobado
CP-05	Contraseñas distintas	Escribir contraseñas que no coinciden.	Se muestra el error 'Las contraseñas no coinciden'.	Aprobado
CP-06	Correo duplicado	Registrar un correo ya existente.	HTTP 400 'Este correo ya está registrado'.	Aprobado
CP-07	Inicio de sesión correcto	Ingresar credenciales válidas.	HTTP 200; se devuelven user_id y name.	Aprobado
CP-08	Inicio de sesión incorrecto	Ingresar credenciales inválidas.	HTTP 401; se solicita revisar la información.	Aprobado
CP-09	Revocación no autorizada	Un editor intenta revocar acceso.	HTTP 403; solo el propietario puede revocar.	Aprobado
CP-10	Sincronización en tiempo real	Editar el documento desde dos clientes.	Ambos clientes reflejan el cambio.	Aprobado

3.3. Evidencias de ejecución

A continuación se incluyen las capturas de pantalla que evidencian el funcionamiento del sistema y la ejecución de las pruebas.

The image displays two screenshots of the CoopDoc web application interface, showing the registration and login processes.

Top Screenshot: Registration Page

- Header:** CoopDoc — Editor de Texto Colaborativo
- Left Panel:**
 - Star icon.
 - Únete junto a tus amigos a CoopDoc**
 - Text: Usando CoopDoc, termina de volada tus trabajos grupales. Para ello, crea una cuenta nueva con nosotros.
 - Links: Ya tengo cuenta, [Regresar a inicio de sesión](#)
- Right Panel:**
 - ¡Bienvenido!**
 - Form fields:
 - Ingresar tu nombre y apellidos: ej. César Ríos Olivares
 - Ingresar tu fecha de nacimiento (dd/mm/aaaa): ej. 17/09/2001
 - Ingresar tu correo de verificación: ej. usuario@correo.com
 - Ingresar tu contraseña: ej. Contraseña!! (with Show button)
 - Verifica tu contraseña: Repite tu contraseña (with Show button)
 - Buttons: Crear cuenta, Registrarse con Google

Bottom Screenshot: Login Page

- Header:** CoopDoc — Editor de Texto Colaborativo
- Left Panel:** (Same as the registration page)
- Right Panel:**
 - ¡Bienvenido!**
 - Star icon.
 - Inicia sesión**
 - Form fields:
 - Ingresar tu correo
 - Ingresar tu contraseña: ej. Contraseña!! (with Show button)
 - Link: [¿Olvidaste tu contraseña?](#)
 - Button: Iniciar sesión
 - Text: ¿No cuentas con cuenta propia?
 - Button: [Regístrate aquí](#)

3.4. Registro de bugs resueltos

Incidencia	Resolución
Los colores y tipografías no cumplían la guía de diseño (cian, negro, verde y fuente Arial).	Se reemplazaron por los tokens hexadecimales oficiales (APP_BACKGROUND, PRIMARY_ACCENT, etc.) y la tipografía Rubik con respaldo del sistema.
La validación de correo era demasiado estricta: exigía la cadena “.com” y rechazaba dominios válidos.	Se sustituyó por una expresión regular completa, coherente con las reglas del PRD (un solo @, sin espacios, sin puntos consecutivos ni en los extremos).
La validación de nombre (^[A-Za-z]+\$) rechazaba acentos y espacios, incluso el ejemplo “César Ríos Oliváres”.	Se relajó la regla para aceptar letras Unicode y espacios, manteniendo el bloqueo de números y caracteres especiales.
El inicio de sesión estaba simulado (éxito codificado) e ignoraba las credenciales.	Se conectó al endpoint real /login, con manejo de los errores 401 y de fallos de conexión.
Uso inseguro de hilos al invocar widgets de tkinter desde un hilo secundario.	Se eliminó dicho patrón; las operaciones de red se ejecutan de forma segura y el servidor corre en un hilo daemon independiente.
El esquema del backend no incluía la fecha de nacimiento solicitada por la interfaz de registro.	Se añadió la columna date_of_birth al modelo y a la tabla, con una migración para bases de datos preexistentes.

3.5. Control de versiones

El versionado siguió un flujo de ramificación en Git: la rama main se mantuvo estable y cada tarea se desarrolló en una rama de funcionalidad (por ejemplo, agregar-login o corregir-validacion), integrada mediante Pull Requests revisados por el equipo. La tabla siguiente resume la evolución del proyecto alineada con los sprints.

Versión	Sprint	Archivos modificados	Descripción del cambio
v0.1.0	Sprint 1	(repositorio, UML, wireframes)	Configuración del repositorio, modelado UML, wireframes en Figma y cronograma del proyecto.
v0.2.0	Sprint 2	backend.py, registro_ui.py, login_ui.py	Base de datos y backend de usuarios; pantallas y lógica de registro e inicio de sesión con validación.
v0.3.0	Sprint 3	coopdoc_app.py	Lienzo de edición: escritura y borrado de texto; formato de negritas e itálicas.
v0.4.0	Sprint 4	coopdoc_app.py	Importación/exportación de .txt; lógica de invitar/revocar colaboradores; canal WebSocket para edición en tiempo real.
v0.5.0	Integración	coopdoc_app.py	Unificación de los tres módulos en coopdoc_app.py (commit de integración) y corrección de errores de UI y validación.
v1.0.0	Sprint 6	coopdoc_app.py,	Estabilización final, documentación técnica y de usuario, y cierre del proyecto.

-o-	Commits on Jun 4, 2026
	Añadir guía de flujo de trabajo con git  laloescar committed 3 days ago
-o-	Commits on Jun 3, 2026
	Eliminación de último archivo .DS_Store restante  laloescar committed 5 days ago
	Eliminar el resto de archivos .DS_Store  laloescar committed 5 days ago
	Ignorar archivos .DS_Store  laloescar committed 5 days ago
	Inclusión de archivo .gitignore  laloescar committed 5 days ago
	Fase alpha de desarrollo. Primer prototipo funcional  laloescar committed 5 days ago
	Eliminar cronograma  laloescar committed 5 days ago
	Unificación de código en un solo archivo .py y organización de archivos en directorios  laloescar committed 5 days ago
	Inicio del repositorio. Incluye documentación del programa, cronograma y primeras versiones aisladas de frontend (registro y login) y backend (servidor y base de datos)  laloescar committed 5 days ago

4. Diccionario Técnico de Clases y Elementos

A continuación se describe la función de cada componente del archivo fuente `coopdoc_app.py`, organizado por capas.

4.1. Constantes y tokens de diseño

- **APP_BACKGROUND, CANVAS_BACKGROUND, PRIMARY_ACCENT, HOVER_ACCENT, TEXT_PRIMARY:** paleta de colores oficial en hexadecimal aplicada a toda la interfaz.
- **API_BASE:** dirección base del servicio local (`http://127.0.0.1:8000`) usada por el cliente.
- **EDITOR_FONT:** tipografía reservada para el lienzo de edición (Open Sans, 12).

4.2. Capa de backend

- **get_db_connection():** abre la conexión a SQLite con `check_same_thread=False` y filas accesibles por nombre.
- **initialize_database():** crea las tablas `users`, `documents` y `permissions` si no existen y migra la columna `date_of_birth`.
- **UserRegister:** modelo Pydantic del registro; concentra los validadores `validate_name`, `validate_dob`, `validate_email`, `validate_password` y `check_passwords_match`.
- **UserLogin, DocumentCreate, InviteUser, RevokeAccess:** modelos de entrada para autenticación, creación de documentos, invitación y revocación.
- **hash_password():** genera el hash SHA-256 de la contraseña.
- **check_permission():** devuelve el rol ('owner' o 'editor') del usuario sobre un documento, o `None` si no tiene acceso.
- **Endpoints:** `health`, `register_user`, `login_user`, `create_document`, `get_document`, `invite_user`, `revoke_access` y `websocket_endpoint`.
- **ConnectionManager:** administra las conexiones WebSocket activas por documento; expone `connect()`, `disconnect()` y `broadcast_text_update()` (estrategia 'el último en escribir gana').
- **start_backend() y wait_until_ready():** levantan Uvicorn en un hilo daemon y esperan a que el servidor responda antes de iniciar la interfaz.

4.3. Validación del cliente

- **validate_registration():** replica en el cliente las reglas de nombre, fecha, correo y contraseña, devolviendo mensajes de error en español antes de enviar la petición.

4.4. Capa de interfaz (frontend)

- **resolve_font_family() e init_fonts():** seleccionan la tipografía Rubik (o un respaldo del sistema) y construyen las fuentes nombradas de la interfaz.
- **PlaceholderEntry:** campo de entrada con texto de marcador de posición en cursiva y soporte de enmascarado de contraseña; expone `value()`, `set_mask()` y `reset()`.
- **make_field(), style_primary(), make_outline_button(), draw_star_logo(), add_hover_underline():** utilidades de construcción de componentes visuales (campos con borde, botones primarios y de contorno, logotipo y enlaces con subrayado al pasar el cursor).
- **App:** controlador principal (hereda de `tk.Tk`); gestiona el cambio entre pantallas con `show()` y el ingreso exitoso con `on_login_success()`.
- **LoginFrame:** pantalla de inicio de sesión; su método `handle_login()` valida y consulta el endpoint `/login`.
- **RegisterFrame:** pantalla de registro a dos columnas; incluye `_build_promo_panel()`, `_build_form_panel()`, `_auto_format_date()`, `_reset_form()` y `handle_register()`.
- **DashboardFrame:** panel posterior al inicio de sesión que da la bienvenida al usuario mediante `greet()`.

5. Documentación para el Usuario

5.1. Manual de usuario final

CoopDoc requiere una instalación previa sencilla para funcionar correctamente. A continuación se describen los requisitos y el uso de la aplicación.

Requisitos previos:

- Python 3 instalado en el equipo.
- Las dependencias del proyecto: fastapi, uvicorn, requests y pydantic.

Instalación de dependencias:

```
pip install fastapi uvicorn requests pydantic
```

Cómo iniciar la aplicación:

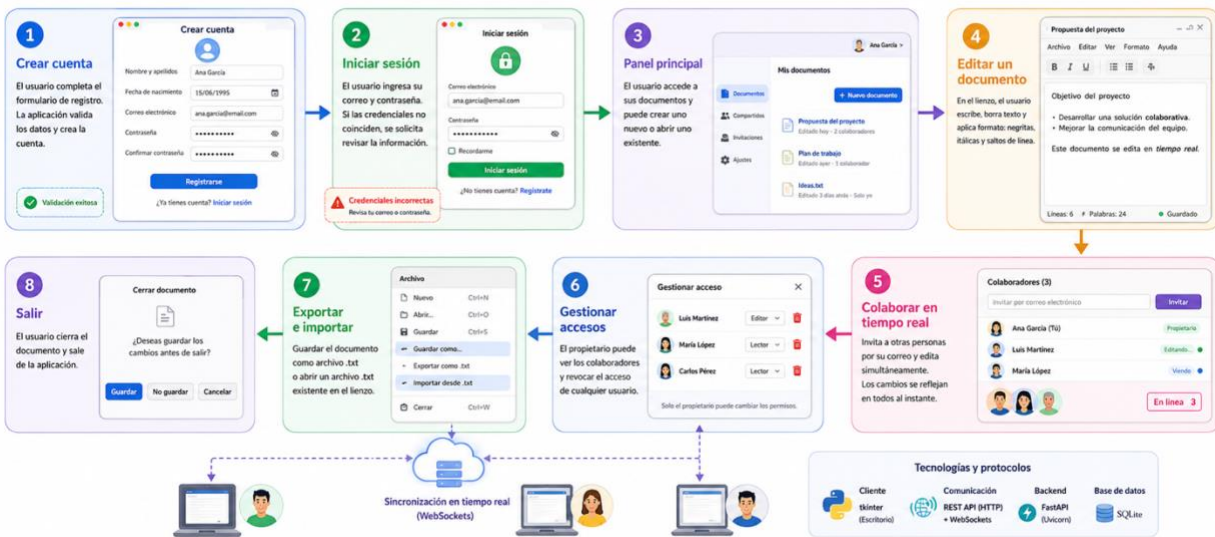
1. Abrir una terminal en la carpeta del proyecto.
2. Ejecutar el comando: `python3 coopdoc_app.py`.
3. Esperar a que aparezca la ventana de bienvenida con la pantalla de inicio de sesión.

Uso de la aplicación:

- **Crear una cuenta:** desde la pantalla de registro, completar nombre y apellidos, fecha de nacimiento, correo y contraseña (con su confirmación). El sistema valida los datos y crea la cuenta.
- **Iniciar sesión:** ingresar el correo y la contraseña registrados. Si las credenciales no coinciden, la aplicación solicita revisar la información.
- **Editar un documento:** en el lienzo, escribir y borrar texto, así como aplicar negritas, itálicas y saltos de línea.
- **Colaborar:** invitar a otras personas por su correo electrónico y editar el documento de forma simultánea; los cambios se reflejan en tiempo real.
- **Gestionar accesos:** el propietario del documento puede revocar el acceso de los colaboradores.
- **Exportar e importar:** guardar el documento en un archivo .txt local o abrir un archivo .txt existente en el lienzo.

Flujo de uso de la aplicación (CoopDoc)

De la autenticación a la colaboración y gestión del documento



5.2. Guía rápida (tutorial básico)

1. Abrir la aplicación y crear una cuenta desde “Regístrate aquí”.
2. Iniciar sesión con el correo y la contraseña recién creados.
3. Crear o abrir un documento desde el panel principal.
4. Escribir en el lienzo y aplicar formato (negritas o itálicas) según se necesite.
5. Invitar a un colaborador por su correo para editar en equipo.
6. Guardar el trabajo exportándolo como archivo .txt.

5.3. Preguntas frecuentes (FAQ)

¿La aplicación no abre? Verificar que Python 3 esté instalado y que se hayan instalado las dependencias con pip install fastapi uvicorn requests pydantic.

¿Aparece un error de conexión con el servidor? Asegurarse de que el puerto 8000 esté libre; la aplicación levanta el servidor localmente al iniciar.

¿Olvidé mi contraseña? La recuperación de contraseña no está disponible en esta versión; se contempla para versiones futuras.

¿Cómo invito a alguien a mi documento? Cualquier usuario con acceso puede invitar a otro indicando su correo electrónico registrado.

¿Quién puede quitar accesos? Únicamente el propietario (creador) del documento puede revocar el acceso de otros colaboradores.

5.4. Glosario

Término	Definición
Lienzo (Canvas)	Área principal donde el usuario escribe y edita el texto del documento.
Colaborador	Usuario invitado que puede editar un documento compartido.
Propietario (Owner)	Usuario creador del documento; es el único que puede revocar accesos.
Editor	Rol con permiso para editar el contenido de un documento.
Permiso	Nivel de acceso de un usuario sobre un documento ('owner' o 'editor').
Token de usuario	Identificador (user_id) devuelto al iniciar sesión, usado en peticiones posteriores.
WebSocket	Canal de comunicación bidireccional que sincroniza la edición en tiempo real.
Sincronización en tiempo real	Reflejo inmediato de los cambios de un usuario en los demás clientes activos.
.txt	Formato de texto plano usado para exportar e importar documentos.
Validación	Comprobación de que los datos cumplen las reglas definidas (nombre, correo, contraseña).

5.5. Video tutorial

Se prevé un material audiovisual de apoyo con los siguientes contenidos en una futura actualización de este documento:

- Instalación de Python y dependencias.
- Registro e inicio de sesión.
- Edición colaborativa y exportación de documentos.

6. Cierre del Proyecto

6.1. Registro de mejoras para futuras versiones

- Añadir más elementos de edición: listas ordenadas y no ordenadas, texto resaltado y color de texto.
- Dar soporte al formato Markdown (.md) para crear, editar, importar y exportar este tipo de archivos.
- Implementar un sistema sencillo de organización de documentos.
- Incorporar las entidades de Comentarios e Historial de versiones previstas en el modelo conceptual.
- Habilitar la recuperación de contraseña y el inicio de sesión con proveedores externos (p. ej., Google).
- Reforzar la seguridad con hashing por sal (bcrypt/argon2) y tokens de sesión.

6.2. Informe de cierre

Objetivos alcanzados:

- Se desarrolló una aplicación de escritorio funcional en Python con arquitectura cliente-servidor.
- Se implementó la autenticación de usuarios con validación estricta en cliente y servidor.
- Se persisten correctamente usuarios, documentos y permisos en SQLite.
- Se estableció la infraestructura de colaboración en tiempo real mediante WebSockets, junto con la gestión de invitaciones y revocaciones de acceso.

Indicadores de éxito:

- La aplicación se ejecuta como un único archivo, integrando cliente y servidor.
- El registro y el inicio de sesión escriben y recuperan datos de la base de datos de forma confiable.
- La interfaz cumple la guía de diseño (paleta y tipografía) y es clara e intuitiva.
- Las reglas de validación se aplican de manera consistente y los errores se comunican con claridad al usuario.